TP Gitlab (4h30)

1. Objectif pédagogique

À l'issue de ce TP, vous serez capable de :

- Comprendre et appliquer le gitflow et le conventional commit
- Mettre en œuvre un cycle de développement collaboratif basé sur les issues et merge requests.
- Concevoir une chaîne d'intégration continue (CI)
- Automatiser les contrôles de qualité et de sécurité
- Automatiser la construction et la conteneurisation de l'application
- Publier automatiquement de la documentation technique (via GitLab Pages).

2. Rendu attendu

À la fin du TP, vous devrez déposer :

Le lien vers le dépôt GitLab public contenant :

- Le projet Python (code source, tests, documentation).
- Le fichier .gitlab-ci.yml décrivant le pipeline complet (build, lint, sécurité, qualité, déploiement de documentation, dockerisation).
- Le Dockerfile utilisé pour conteneuriser l'application.
- L'ensembre des fichiers complémentaires nécessaire (configuration de lint, dépendances, etc.).

Une explication détaillée (dans un document ou un fichier REPORT.md dans le dépôt) décrivant :

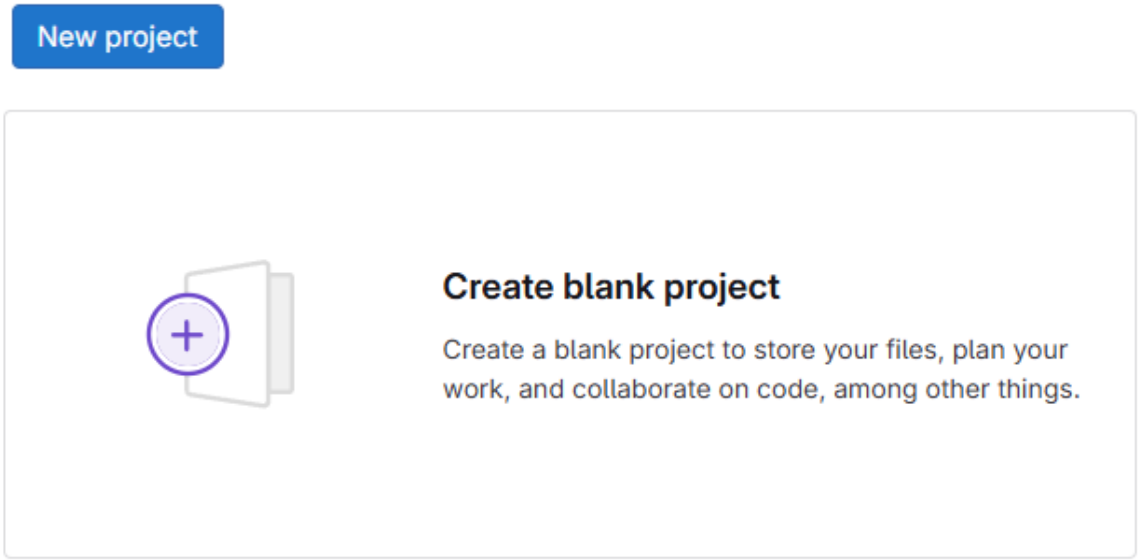
- Mise en place du projet GitLab
- Création du dépôt public.
- Configuration des accès (SSH).
- Mise en place des branches (main, develop, feature/...) selon le GitFlow.

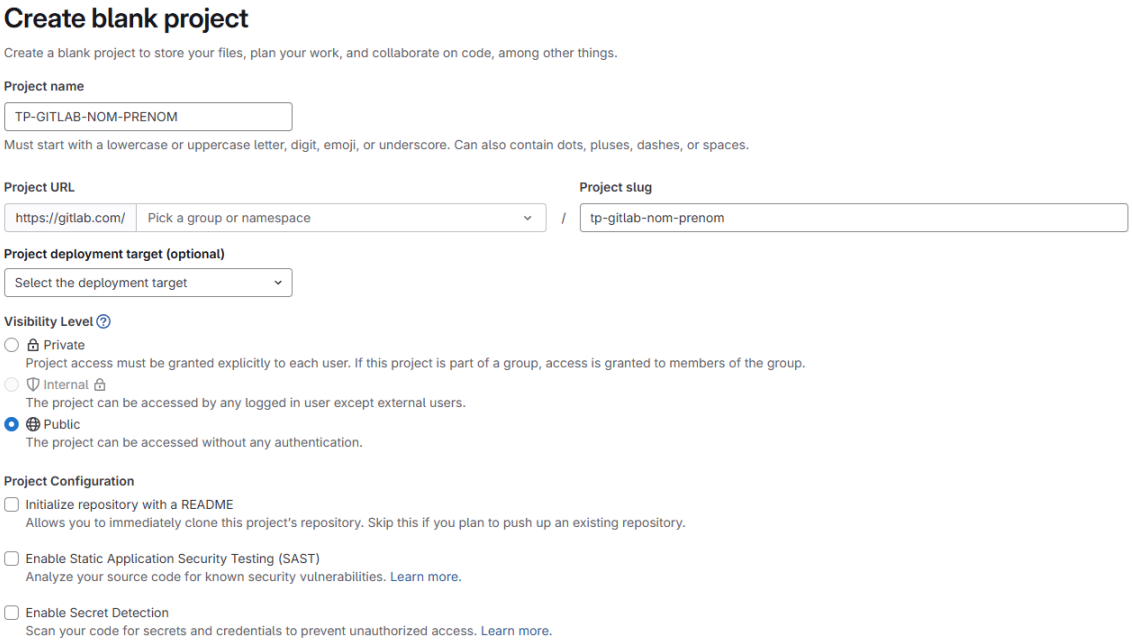
Pipeline CI/CD

- Explication du contenu du fichier .gitlab-ci.yml.
- Description de chaque étape : build, lint, security, quality, deploy.
- Outils choisis (pytest, pylint, bandit, snyk, etc.) et leur rôle.

3. Mise en place du projet Gitlab

Afin de réaliser ce TP, vous devez vous rendre sur gitlab.com et vous créer un compte. Créer un nouveau projet : https://gitlab.com/dashboard/projects avec la configuration suivante :





Suivez ensuite les instructions afin de cloner votre dépôt soit en SSH soit en HTTPS

4. Mise en place d'un projet Python

Une archive d'un projet Python (une todo list) est disponible sur l'ENT. Elle contient :

- un dossier src/ avec du code de base,
- un dossier tests/ avec quelques tests unitaires,
- un fichier requirements.txt pour les dépendances,
- un fichier .pylintrc pour la configuration de l'analyseur de code.

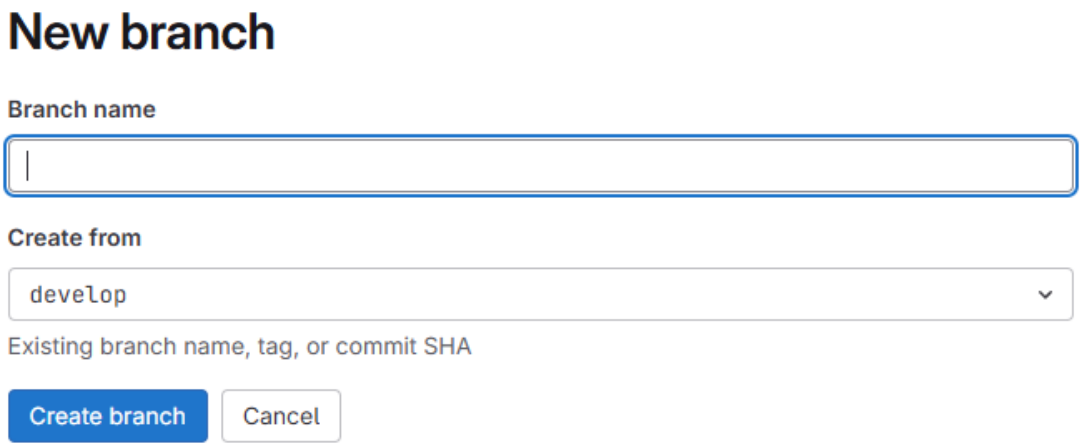
Liste des tâches à réaliser :

Dezipper le fichier et mettez-le sur votre projet gitlab. Vous pouvez seulement pour le premier commit mettre directement sur la branche main pour la suite vous devrez respecter le **gitflow** ainsi que le **conventional commit** vu en cours.

Ajoutez un .gitignore adapté à Python.

Mettez en place la stratégie de branches GitFlow :

- main : branche stable qui servira de base de production,
- develop : branche d'intégration,
- feature/... : branches pour chaque nouvelle fonctionnalité.



Aide :

Documentation sur les commandes git de base.

https://education.github.com/git-cheat-sheet-education.pdf

Documentation GitLab pour la gestion des dépôts et des branches :

https://docs.gitlab.com/ee/gitlab-basics/

Modèle .gitignore Python :

https://github.com/github/gitignore/blob/main/Python.gitignore

5. Mise en place de l'intégration continue

Une fois votre projet Python importé dans GitLab, vous allez configurer un fichier .gitlab-ci.yml à la racine du dépôt.

Ce fichier décrit votre pipeline et les étapes qui seront exécutées automatiquement à chaque commit ou merge request.

Étapes attendues dans votre pipeline :

- Build : vérifier que le projet se compile correctement.
- Lint : vérifier la qualité du code avec un outil adapté.
- Sécurité : analyser le code source pour détecter d'éventuelles failles.
- Qualité : exécuter les tests unitaires et générer un rapport.
- Déploiement de la documentation : publier automatiquement une documentation simple (ex. via pydoc ou sphinx).

Vous êtes libres de choisir vos outils, mais voici quelques suggestions :

- Tests unitaires : pytest
- Sécurité : bandit, snyk
- ualité de code : pylint, flake8

Quelques commandes utiles

À tester d'abord en local (IDE ou terminal), avant de les intégrer dans le pipeline :

```
pip install --upgrade pip
pip install -r requirements.txt

# Compilation des fichiers
python -m py_compile src/todo_model.py
python -m py_compile src/todo_gui.py
python -m py_compile console_app.py
python -m py_compile main.py

# Lancement des tests unitaires
python -m pytest tests/

# Analyse sécurité avec Bandit
bandit -r src/ -f json -o bandit-report.json || true
stages:
```

Conseils

Testez chaque commande manuellement avant de l'intégrer dans le pipeline.

Installez les dépendances nécessaires (tests, lint, sécurité) dans vos jobs CI.

Utilisez l'éditeur de pipeline intégré à GitLab pour vérifier votre .gitlab-ci.yml.

Faites évoluer votre pipeline par itérations (commencez simple, ajoutez les étapes une par une).

Aide :

Documentation de gitlab CI

https://docs.gitlab.com/ci/

Syntaxe de base :

```
stages:
  - build
  - lint
  - security
  - quality
  - deploy

build-job:
  stage: build
  image: python:3.10
  script:
    - pip install --upgrade pip
    - pip install -r requirements.txt
    - python -m py_compile main.py
```

6. Build et containerisation

Vous allez maintenant construire et publier l'image Docker de votre application sur DockerHub à chaque mise à jour de votre projet.

Étapes à suivre

Création d'un compte DockerHub

Si vous n'en avez pas déjà un, créez un compte gratuit sur https://hub.docker.com

Créez un repository public (ex. tp-gitlab-votrelogin).

Création d'un token d'accès

Dans DockerHub, allez dans Account Settings → Security → New Access Token.

Donnez-lui un nom explicite (ex. gitlab-ci-token).

Copiez le token généré (Attention celui-ci n'apparaît qu'une seule fois).

Ajout de secrets dans GitLab

Allez dans votre projet GitLab → Settings → CI/CD → Variables.

Ajoutez les variables suivantes

DOCKERHUB_USERNAME : votre login DockerHub

DOCKERHUB_TOKEN : le token d'accès créé à l'étape précédente

Configuration du pipeline

Ajoutez un job dans votre .gitlab-ci.yml pour :

- Se connecter à DockerHub avec les secrets
- Construire l'image
- La pousser sur DockerHub